

Dependency Injection

Dependency Injection lets a class **ask** for what it needs instead of building it. Angular's injector then hands over a shared or a fresh instance depending on **where you provided it**. This guide covers **inject()**, scopes, the injector hierarchy, provider recipes, and testing.

1 · THE PROBLEM

Why a component should not build its own dependencies

Imagine `CartComponent` writes `const cart = new CartService()`. Three problems appear at once:

- **No sharing.** The header badge creates *another* `CartService`, so it never sees the same items.
- **Hard to test.** You cannot swap the real service for a fake.
- **Tight coupling.** If `CartService` later needs an `HttpClient`, every place that wrote `new` must change.

Dependency Injection (DI) flips this around: a class just *declares what it needs*, and Angular's **injector** supplies it — creating it once and sharing it, or creating a fresh one, depending on where it was provided.

Component asks → `inject(CartService)` → Injector looks it up / creates it → Component receives it

2 · THE BASICS

@Injectable and inject()

```
@Injectable({ providedIn: 'root' }) // one instance for the whole app
export class CartService {
  readonly items = signal<number[]>([]);
  add(id: number) { this.items.update((l) => [...l, id]); }
}

@Component({ selector: 'app-badge', template: `{{ cart.items().length }}` })
export class BadgeComponent {
  readonly cart = inject(CartService); // modern style
  // constructor(private cart: CartService) {} // older, equivalent style
}
```

`providedIn: 'root'` makes the service **tree-shakable**: it is created lazily the first time something injects it, and removed from the bundle if nothing ever does.

RULE

`inject()` only works in an **injection context**: a constructor, a field initializer, or a function Angular runs for you (a guard, a resolver, a provider factory). Calling it inside an event handler throws **NG0203**. Outside Angular's own code, use `runInInjectionContext`.

3 · WHERE TO PROVIDE IT

The place you provide a service decides its scope

WHERE	INSTANCES	LIVES UNTIL
<code>providedIn: 'root'</code>	One for the whole app	The app closes
<code>bootstrapApplication(..., { providers })</code> / <code>app.config.ts</code>	One for the whole app	The app closes
A route's <code>providers: [...]</code>	One per route injector (shared by that route and its children)	The route injector is destroyed
A component's <code>providers: [...]</code>	One per component instance (and its children)	That component is destroyed
A component's <code>viewProviders: [...]</code>	Like <code>providers</code> but hidden from projected content	That component is destroyed

```
@Component({
  selector: 'app-row-editor',
  providers: [RowEditState], // every <app-row-editor> gets its own RowEditState
  template: `...`,
})
export class RowEditorComponent { readonly state = inject(RowEditState); }
```

TIP

Try it live in the Angular Lab **Dependency Injection** lesson: two components share a root service (same instance number), and two components with `providers` each get their own.

4 · HOW ANGULAR FINDS IT

The injector hierarchy

When a class asks for a service, Angular searches upward until it finds a provider:

- 1 **Element injectors** — the component and its parents in the template (`providers`, `viewProviders`, directives).
- 2 **Environment injectors** — the route injector(s), then the **root** injector (`providedIn: 'root'`, `app.config`).
- 3 **Platform injector**, and finally the **NullInjector**, which throws `NullInjectorError: No provider for X`.

INJECT() OPTION	MEANING
<code>inject(X, { optional: true })</code>	Return <code>null</code> instead of throwing when there is no provider
<code>inject(X, { self: true })</code>	Only look at this element's own injector
<code>inject(X, { skipSelf: true })</code>	Skip this element, look at its parents (e.g. get the <i>parent's</i> instance)
<code>inject(X, { host: true })</code>	Stop searching at the host component

5 · PROVIDER RECIPES

Telling Angular how to build something

```
providers: [  
  { provide: Logger, useClass: ConsoleLogger },           // build a different class  
  { provide: API_URL, useValue: 'https://api.example.com' }, // a ready-made value  
  { provide: Clock, useFactory: () => new Clock(inject(Config).timezone) }, // compute it  
  { provide: OldLogger, useExisting: Logger },             // an alias to another provider  
]  
  
// Not a class? Use an InjectionToken as the key  
export const API_URL = new InjectionToken<string>('API_URL', {  
  providedIn: 'root',  
  factory: () => 'https://api.example.com', // a default; override in providers  
});  
const url = inject(API_URL);
```

Add `multi: true` when many providers should contribute to **one list** under the same token (for example a set of validators or plugins).

6 · TESTING

The best reason to use DI

```
TestBed.configureTestingModule({  
  providers: [{ provide: CartService, useValue: { items: signal([1, 2]) } }],  
});  
const fixture = TestBed.createComponent(BadgeComponent);  
fixture.detectChanges();  
expect(fixture.nativeElement.textContent).toContain('2'); // used the fake
```

Because the component only asks for `CartService`, the test can hand it any stand-in — no network, no real state, no changes to the component.

7 · COMMON MISTAKES

Things that trip people up

- **Providing the same service twice.** A service that is `providedIn: 'root'` and listed in a component's `providers` becomes **two different instances**. State seems to "not update" because the two sides hold different objects.
- **Forgetting `@Injectable()`**. A service that injects other services needs the decorator.
- **Circular dependencies.** A needs B and B needs A. Break the cycle by extracting shared logic into a third service.
- **Calling `inject()` too late** (inside a click handler). Store the injected value in a field instead.
- **Putting mutable per-user data in a root service on the server.** With SSR a root service is shared between requests; keep user data out of it.

8 · CHEAT SHEET & QUICK QUIZ

I WANT...	DO THIS
One shared instance for the app	<code>@Injectable({ providedIn: 'root' })</code>
A new instance per component	<code>providers: [Service]</code> on the component
To use a service	<code>private readonly x = inject(Service)</code>
A config value or interface as a dependency	<code>InjectionToken</code> + <code>useValue</code> / <code>useFactory</code>
A fake in a test	<code>{ provide: Service, useValue: fake }</code>

8 · CHEAT SHEET & QUICK QUIZ · CONTINUED

Q: What does `providedIn: 'root'` give you?

A single app-wide instance, created lazily and tree-shaken if unused.

Q: When put a service in a component's providers?

When each component (copy) needs its own private state that dies with it.

Q: Why does a service sometimes seem not to share state?

It was provided in more than one place, so there are several instances.

Try it live: youssefezzat17.github.io/angular-lab/di